

# APEX 4.37.1

## 用户意图驱动的全栈交付系统 产品设计、技术架构与运行治理白皮书

| 属性名称  | 属性值                        |
|-------|----------------------------|
| 正式版本  | 4.37.1                     |
| 文档状态  | 独立对外说明；与 APEX Core 发布同步    |
| 覆盖范围  | 产品设计、UI/UX、前端、后端接口、验证、运行治理 |
| 唯一主目录 | ~/codex/apex/APEX          |
| 发布日期  | 2026-09-20                 |

阅读说明：本文件完整说明 APEX 4.37.1  
的目标、流程、视觉冻结、工程实施、验收、接入与治理；它不要求读者跳转到其他文档才能理解核心机制。

# 执行摘要

APEX 是将不完整产品需求转化为可确认、可实现、可验证交付的企业级 Agentic Delivery System。它统一产品分析、视觉设计、Stitch 可编辑画布、前后端实施和真实验收，解决 AI 交付常见的跳流程、视觉偏差、Existing 系统脱离事实、会话串扰和证据缺失问题。

本白皮书面向业务负责人、产品与设计负责人、架构与研发负责人、平台治理与交付负责人。读者可据此判断 APEX 的适用范围、系统边界、实施路径、质量门和治理责任。

- 1. 业务价值：把模糊需求转化为经确认的 Delivery Contract，减少重复澄清与返工。
- 2. 体验价值：以运行时 Demo、Stitch 和设计契约共同冻结视觉事实，缩小从设计到代码的偏差。
- 3. 工程价值：用 Router、授权、run/session 隔离和真实 Gate 3 证据约束交付过程。
- 4. 治理价值：将版本、文档、PDF、发布审计和全局 Skill 同步纳入可复核的发布闭环。

## 问题陈述、受众与非目标

企业级交付常见的问题是：业务描述短而不完整，设计与代码分别演进，Existing 改造没有读取真实系统，会话可能复用错误上下文，最终验收只有页面截图而没有功能、性能、无障碍和接口证据。APEX 的设计目标是减少这些断点，而不是代替业务负责人决定业务优先级。

- 1. 业务负责人：确认范围、风险、质量门与 Gate 决策。
- 2. 产品与设计负责人：确认信息架构、任务流、视觉方向、内容与体验边界。
- 3. 研发与架构负责人：确认项目事实、接口、权限、实施映射和工程约束。
- 4. 平台与交付负责人：确认治理、会话隔离、发布流程、审计与恢复。
- 5. 非目标：APEX 不充当业务生产系统、部署平台、通用后端框架或仅提供静态设计稿的素材库。

## 产品简介与系统边界

APEX 是一个技术型交付编排产品，不是单一 Agent、设计工具或代码生成器。它位于用户需求、项目真实系统、设计工具、工程工具和验收工具之间：向上接收自然语言与组织规范，向下以可审计的 run 驱动产品分析、设计冻结、实现与验证。

| 需求 | 提炼、冲突识别、合同、确认与追踪 | 替业务方决定商业优先级或不可逆业务决策 |
|----|------------------|---------------------|

| 能力项 | 能力项描述                      | 能力项输出              |
|-----|----------------------------|--------------------|
| 设计  | 运行时 Demo、Stitch、设计契约、保真校验  | 将未确认的审美偏好视为最终规格    |
| 工程  | 实施映射、受控动作、验证计划与证据          | 替项目托管生产业务运行时       |
| 治理  | run/session 隔离、审批、授权、审计、恢复 | 绕过宿主权限、组织安全制度或发布责任 |

## 产品功能体系

APEX 功能按“准入—设计—实施—验证—治理”划分，每项能力均有输入、状态、输出与证据，不以聊天文本作为唯一事实。

| 能力项         | 能力项描述                                                      | 能力项输出                                                                |
|-------------|------------------------------------------------------------|----------------------------------------------------------------------|
| 需求准入        | 短需求提炼、轨道识别、范围与风险判断                                         | Intent Brief、Delivery Contract、Gate 1                                |
| Existing 勘测 | 代码、路由、页面、接口、规范与保护边界读取                                      | Project Inventory、Existing Baseline、Functional Freeze                |
| 视觉工程        | 视觉方案确认后自动生成并登记运行时 Demo，不追加 Demo 确认；其后 Stitch 单独确认或进入直接代码路线 | Visual Reference、直接代码实施方案或 Stitch Freeze、Site Contract、Visual Bundle |
| 实施编排        | 设计节点到组件、API、权限、事件与测试映射                                     | Implementation Map、Dependency Lock、受控实施授权                            |
| 质量验证        | 浏览器、契约、视觉、无障碍、性能、回归验证                                      | Verification Plan、Evidence、Verification Bundle、Gate 3                |
| 运行治理        | Router、审批、session、授权、lease、恢复、版本发布                         | Run State、事件、授权回执、Checkpoint、Release Audit                           |

# 产品架构：从需求到可验证交付

产品架构将输入、编排能力与交付结果分开表达。它强调每一个阶段都既有专业角色的判断，也有可被机器检查的工件和证据。



图示说明：APEX 不是替代任一专业角色，而是在关键确认点将产品、视觉、工程和验证组织为一套可追踪的交付系统。

图 1. APEX 产品架构。输入既包括自然语言需求，也包括组织规范和项目真实事实；输出是经 Gate 确认的可交付结论。

# 逻辑技术架构

APEX 采用控制面与执行面分离的架构。控制面定义版本、协议、状态机、Gate、授权、能力注册和发布；执行面读取目标项目事实并调用浏览器、Stitch、测试、构建与项目工具。两者通过 Router 和 Action Gateway 连接。

| 接入层   | Codex 全局 apex Skill、Preflight、Invocation Spec                      | 识别 UI/UX/可见交付任务，定位唯一 Core，不裁决 Gate            |
|-------|--------------------------------------------------------------------|-----------------------------------------------|
| 控制面   | APEX Router、Run Controller、Version Policy                          | 创建项目 run，绑定 session，返回 allowedActions，维护状态与版本 |
| 安全执行面 | Action Gateway、Runtime Guard、Mutation Lease                        | 验证动作授权与 state hash，只运行注册脚本，阻止并发项目写入           |
| 能力面   | Intake、Baseline、Visual、Stitch、Implementation、Verification、Recovery | 按阶段产出结构化事实与证据                                 |
| 项目面   | 代码、页面、接口、规范、/.apex                                                 | 保存项目真实事实及运行产物；Core 不保存项目数据                    |

# 运行时状态、数据与证据模型

每个项目调用生成一个独立 run。Run State 是阶段与 Gate 的机器真相源；所有关键产物均以 run 相对路径和哈希连接，状态变化使旧授权失效。

| Project Identity      | projectId、projectRoot                            | 首次接入建立，阻止项目混用                    |
|-----------------------|--------------------------------------------------|----------------------------------|
| Run State             | track、scope、phase、gates、locks、revision、artifacts | Router 内部控制；每次状态写入递增 revision    |
| Approval Receipt      | gate、session、工件路径与 SHA-256                       | Gate 1 / 视觉方案 / Stitch / 实施确认时冻结 |
| Authorization Receipt | 项目、run、session、action、stateHash、过期时间             | 动作执行前签发；状态变化即失效                  |
| Evidence Bundle       | 浏览器、契约、视觉、无障碍、性能、回归证据                            | Gate 3 前汇集；缺项不能自动通过              |

# 证据与可追溯性：同一 run 的闭环

高质量交付不仅需要结果，还必须能解释结果的来源、确认过程、实现映射和真实运行证据。以下闭环用于避免把截图、聊天记录或单次测试当作唯一事实。



图 2. APEX 将事实、确认、实现与验证写入同一 Run State；状态 revision 或工件哈希变化会使依赖该结论的授权失效。

# 端到端工作流与关键决策

完整交付按以下顺序运行。流程允许恢复和局部重算，但不允许为了节省步骤而跳过事实、确认或证据。

|                | 事实与证据                                  | 关键决策                                                           |
|----------------|----------------------------------------|----------------------------------------------------------------|
| 1. 准入          | 创建新 run，识别 Greenfield / Existing、范围、授权 | Router 校验 session；不复用跨 session run                             |
| 2. 分析          | 提炼意图、收集规范；Existing 读取真实基线              | 影响业务语义、权限、外部依赖的事项请求确认                                          |
| 3. Gate 1      | 冻结需求、质量、风险、Existing 保护边界               | 用户确认后才允许正式视觉                                                   |
| 4. 视觉          | 确认视觉描述与实施方案后直接生成并登记运行时 Demo            | 运行时 Demo 登记后才选择 Stitch 的独立确认或直接代码；两者均冻结同一运行时 Demo；变化撤销旧 Gate 2 |
| 5. Gate 2 / 实施 | 编译 Visual Bundle、实现映射与依赖锁              | 机器验证通过后开放项目写入授权                                                |
| 6. 验证 / Gate 3 | 执行真实浏览器、契约、质量与回归证据                     | 证据完整且通过才形成交付结论                                                 |

## 1. 系统定位与价值

APEX 是面向产品、设计、研发与交付团队的 Agentic Delivery System。它将用户不完整的自然语言需求，转化为可确认的交付合同，再将视觉事实、代码实现与真实验收证据连接为可恢复、可审计的闭环。

它不把“生成页面”视为交付完成，而把产品意图、Existing 系统事实、运行时 Demo、Stitch 画布、设计契约、实施映射与 Gate 3 证据的一致性视为完成条件。

- 1. 防止短需求直接跳到代码，先形成事实、假设、范围、风险与质量合同。
- 2. 防止设计稿与落地代码分离，通过运行时 Demo、Stitch、DESIGN.md 和 Implementation Map 形成追踪。
- 3. 防止会话中断、并发改造和旧上下文造成跳流程，通过 Router、run、session、审批和授权治理。
- 4. 支持 Greenfield 0-1 落地与 Existing 系统迭代改造，不牺牲真实业务边界。

## 2. 适用场景与范围分级

APEX 的入口判断不依赖用户是否写出完整 PRD 或关键词。只要任务涉及可见页面、UI、UX、视觉、交互、响应式、无障碍、截图还原或页面验收，即应进入 APEX。

| 起点   | 用户意图、业务目标、领域与体验方向              | 真实项目代码、页面、路由、接口、已有规范与保护边界                             |
|------|--------------------------------|-------------------------------------------------------|
| 强制前置 | Intent Brief、Delivery Contract | Project Inventory、Existing Baseline、Functional Freeze |
| 视觉前提 | Gate 1 产品与质量确认                 | 先完成只读基线；否则不得正式出运行时 Demo或 Stitch                       |
| 交付结果 | 新能力的前后端实现与验证                   | 保留既有事实并完成可追踪的改造与回归                                    |

Lite 适合单模块或单交互点；Standard 适合单页整改与局部系统统一；Full 适合整站、页面家族、前后端与视觉系统级交付。范围影响产物深度，不允许降低 Gate 语义。

## 2.1 双轨判断：Greenfield 与 Existing

双轨不是用户主观选择的标签，而是 Router 在准入期根据项目事实完成的硬判断。只要待交付能力需要读取、保留、替换或验证一个既有系统中的任意事实，就进入 Existing 轨道；只有不存在可复用的既有系统事实时，才进入 Greenfield 轨道。

| 存在代码仓库、可运行页面、路由、接口、数据模型或已有设计系统       | Existing   | 先做只读 Inventory、页面/接口/权限勘测与 Existing Baseline；未完成前不得正式出运行时 Demo。 |
|--------------------------------------|------------|-----------------------------------------------------------------|
| 需求包含“改造、优化、统一、保留、兼容、修复、迁移、在原系统上增加”   | Existing   | 冻结保护边界、回归路径和不可变业务规则；视觉与实现均须引用真实基线。                              |
| 只有业务目标、用户问题、领域信息或空项目脚手架，且无可读取的既有业务事实 | Greenfield | 建立 Intent Brief、领域假设、信息架构与 Delivery Contract。                   |
| 既有系统存在，但本次要独立建设新产品                   | 双轨并行       | Existing 作为迁移/集成边界读取；新产品按 Greenfield 形成信息架构，两个范围分别确认。           |

## 2.2 场景判断：用户到底要交付什么

场景判断将一句自然语言请求转为交付类型、证据类型和调用能力，避免把所有请求误处理成“设计一张页面”或“生成一段代码”。一个需求可同时命中多个场景，但必须明确主场景和依赖场景。

| 产品能力          | 交付物                  | 验证证据                                          |
|---------------|----------------------|-----------------------------------------------|
| 产品能力从 0 到 1   | 做一个系统 / 新增产品 / 从想法落地 | 角色、任务、领域模型、信息架构、页面族、接口契约与端到端验证。               |
| Existing 体验改造 | 改造现有系统 / 高级专业 / 统一视觉 | 代码和页面基线、保留边界、运行时 Demo、Stitch、视觉差异与页面族回归。      |
| 单页面或组件优化      | 优化报表 / 表单 / 导航 / 交互  | 问题定位、组件状态、响应式、无障碍、交互和视觉回归。                    |
| 功能与业务流程建设     | 增加审批、权限、数据、工作流、AI 能力 | 功能拆解、领域对象、状态转换、角色权限、API Contract、异常路径与真实运行证据。 |
| 数据叙事与报告       | 成长报告 / 看板 / 数据分析     | 指标口径、趋势、异常、目标、预测、AI 解读、建议、图表可访问性与数据来源。        |

### 2.3 规模判断：决定工作深度，而非省略质量

规模判断用于选择 Lite、Standard 或 Full 的产物深度、验收覆盖和 token 预算。它不允许跳过 Gate，只决定一次需要勘测多少页面、设计多少状态、验证多大范围。

| 规模       | 交付物                               | 验证证据                                             |
|----------|-----------------------------------|--------------------------------------------------|
| Lite     | 1 个页面或组件；1 个角色；无新增跨服务依赖；低风险       | Intent 摘要、影响范围、必要视觉/交互、关键路径测试与针对性证据。             |
| Standard | 单个页面族或模块；2-3 个角色/状态；已有接口或局部新增接口   | 完整 Gate 1/2/3、Existing Baseline、设计契约、实施映射、页面族回归。 |
| Full     | 多页面、多角色、领域状态机、跨端、外部集成、迁移或高风险数据/权限 | 全量产品模型、服务边界、依赖锁、代表页 Proof、性能/无障碍/安全/回归证据。        |

2.4 功能解析与拆解：把“想要”变为可落地能力

APEX 的功能解析不止列功能清单，而是从用户目标反向拆到可设计、可实现、可验证的最小能力单元。每个单元必须能回答：谁在什么条件下做什么、读写什么数据、状态如何变化、受什么权限约束、失败时如何恢复，以及怎样证明已经完成。

| 业务目标与用户任务 | 能力与流程                       | 领域与数据                                                          | 体验与设计 | 工程与验收 |
|-----------|-----------------------------|----------------------------------------------------------------|-------|-------|
| 业务目标与用户任务 | 目标、用户角色、触发条件、成功标准、禁止行为      | Outcome、Actor、Job、优先级、假设与风险。                                   |       |       |
| 能力与流程     | 主路径、分支、异常、人工介入、通知与审批        | Capability Map、Journey、状态转换与 Acceptance Criteria。              |       |       |
| 领域与数据     | 实体、字段、来源、口径、生命周期、隐私与保留规则    | Domain Model、Data Contract、指标定义与权限矩阵。                          |       |       |
| 体验与设计     | 信息层级、页面族、组件状态、空态、加载态、响应式与动效 | IA、Screen Map、Stitch、Token、Site Contract。                      |       |       |
| 工程与验收     | 前后端边界、API、事件、权限、测试与回归范围     | Implementation Map、API Contract、Test Plan、Verification Bundle。 |       |       |

# 准入规则可视化：双轨、场景与规模

图 3 把准入阶段的判断规则外显为一个可复核的决策模型。它避免仅凭“做一个页面”或“优化一下”就跳入某种固定工作流。



图 3. 准入输出决定后续所需的事实勘测、设计深度、实施授权与验收覆盖；规模越大，不是流程越少，而是证据范围越完整。

# 能力拆解可视化：从用户目标到验证交付

图 4 将功能解析连接到设计和工程实现。其作用是把模糊愿望转化成可以在 Gate 2 映射、在 Gate 3 验证的能力单元。



图 4. 功能拆解的每一层都留下可追溯工件；任何未定义的角色、数据、状态、权限或异常路径都不能被“页面已生成”掩盖。

### 3. 一句话需求、体验策略与候选评选

用户可以只说“基于参考图生成一个高级、专业、效果炫酷的运行时 Demo”或“改造这个系统”。APEX 不把这些形容词直接翻译成模板：先由设计导演将其拆解为可比较、可确认、可验证的体验策略；策略通过质量门后才允许生成候选运行时 Demo。

- 1. 提炼业务目标、用户、关键任务、页面族、数据、权限、风险、已有事实与未知项。
- 2. 将“高级、专业、华丽、炫酷、动效拉满”等偏好转为信息层级、构图、字重、密度、色彩、材质、动效节奏、交互反馈和禁止模式。
- 3. 识别会改变业务语义、权限、外部依赖或不可逆迁移的事项，形成备选与确认点；其余内容以明确假设推进。
- 4. 输出 Intent Brief、Delivery Contract、Experience Strategy 与质量门槛；策略必须覆盖信息层级、数据问题/编码/交互、功能状态/事件/验收、视觉系统、响应式、无障碍、动效与反模式。
- 5. Experience Evaluator 对当前策略执行 85/100 阈值和关键缺失检查；不通过时回到文案拆解，不得生成正式运行时 Demo。
- 6. Gate 1 后在视觉实施方案中比较信息架构与数据表达取舍，并明确所选方向、真实来源与参数；用户确认完整视觉方案后，系统按该唯一方案生成并登记运行时 Demo。

### 4. 质量定义与专业标准

APEX 以标准校准而非自创形容词标准。项目可追加公司、行业或职业规范，但必须将它们转为可验证的 Contract、Freeze 或 Evidence。

| 质量属性  | 标准与规范                                                       |
|-------|-------------------------------------------------------------|
| 需求与追踪 | ISO/IEC/IEEE 29148：Intent Brief、Delivery Contract、假设、冲突与确认项 |
| 架构    | ISO/IEC/IEEE 42010：系统分层、关注点、Connector、Adapter 与架构描述         |
| 质量    | ISO/IEC 25010：可用性、可靠性、兼容性、性能、安全与可维护性合同                      |
| 无障碍   | WCAG 2.2 AA：真实页面零遗留违规证据                                     |
| 性能    | 默认 LCP ≤ 2500ms、INP ≤ 200ms、CLS ≤ 0.1，并记录测量环境               |
| 安全开发  | NIST SSDF：权限、依赖、接口和交付前安全治理输入                                |

## 5. 三个 Gate 与交付状态机

Gate 是用户确认与机器验证的组合，不是单纯的流程提示。状态变更必须经 Router，且任何视觉、接口、领域或运行时变化都会撤销受影响的后续证据。

|               | 用户确认                                                          | 机器验证                                                                                                                         |
|---------------|---------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Gate 1        | 目标、范围、数据、风险、质量、假设与体验策略                                        | Intent Brief、Delivery Contract、Experience Strategy 与通过的 85 分质量证据；Existing 另需 Inventory、Baseline、Functional Freeze；通过后才允许候选视觉 |
| Visual 确认     | 需求拆解、视觉描述、布局、色彩、组件、动效、在线候选与待安装依赖；方案确认后直接生成并登记运行时 Demo         | Visual Execution Plan 是唯一视觉确认；运行时 Demo 工件必须锁定官方来源、精确版本、API、风险与 Gate 2 后安装计划                                                  |
| Stitch / 直接代码 | 运行时 Demo 登记后选择：Stitch 内容与画布单独确认；或以同一运行时 Demo、来源、代码目标与实施方案直接实施 | Stitch 须有候选选择哈希、Visual Reference、Stitch Freeze 与严格保真证据；直接代码路线须有登记运行时 Demo 与实施方案。两者均不得跳过确认                                    |
| Gate 2        | 所选视觉基线、设计契约、依赖与实现映射                                           | Site Contract、Visual Bundle、Implementation Map、Dependency Lock，以及与当前实施基线一致的严格校验；通过后开放实施                                      |
| Proof         | 代表页或高风险流程的可实现性                                                | 真实实现、浏览器、结构与合同证据                                                                                                             |
| Gate 3        | 真实交付结果                                                        | Verification Bundle：功能、视觉、无障碍、性能、契约、回归，及多视口、默认/加载/空/异常/无权限状态与关键交互的稳定性矩阵                                                      |

# workflow总览：从短需求到 Gate 3 交付

下图展示 APEX 的主交付路径。所有场景先完成需求分析、Existing 基线 ( 如适用 ) 、视觉描述与实施方案；视觉方案确认后直接生成并登记严格运行时 Demo，不增加运行时 Demo人工确认。工件齐全后，用户才选择进入独立的 Stitch 确认或直接代码实施方案确认；两者均保留视觉冻结、工程实施和真实验证。



# 全链路审核门：确认、冻结与交付

本图将用户确认点与机器 Gate 明确分层。确认节点决定“做什么、按什么视觉与依赖方案做”；Gate 2 决定“是否允许写代码和安装已批准依赖”；Gate 3 决定“是否能以真实证据交付”。



图 6. 用户确认最多四类，其中 Stitch 为条件式确认；任何可见调整都会撤销受影响的下游结论。

# 技术架构总览：从宿主到项目证据

APEX 将 Skill 的发现能力、Router 的流程裁决、Action Gateway 的受控执行与项目真实系统分层隔离。Markdown 定义策略与标准，代码负责不可绕过的状态、授权和并发控制。



图 6. 更新路径：Core 更新 → 版本策略 → 文档与 PDF 重建 → 发布审计 → 下一次 Router 调用加载最新 Bridge。

## 6. 视觉真相源、确认顺序与 Stitch

视觉不是统计卡片堆叠或后台管理模板的同义词。APEX 可根据产品任务采用故事化叙事、杂志化编排、现代极简、强动效或其他经确认的视觉方向；关键在于先将参考图和用户意图拆解为体验策略、视觉描述和实施方案，并把 Visual 与 Stitch 的确认清晰分开。

1. 固定前置顺序为：一句话需求/参考图 → 文案与体验策略 → 策略质量评分 → Gate 1 确认 → 视觉效果描述与实施方案确认 → 直接生成并登记运行时 Demo → Stitch 确认或直接代码实施方案确认。不得直接从需求跳到实现。
2. 视觉方案确认后必须按已确认的唯一方案自动生成运行时 Demo，并完整登记来源、参数与运行时证据；不追加运行时 Demo 确认。登记后才允许选择进入 Stitch 或直接代码路线。
3. 直接代码路线不生成 Stitch，但必须将已登记运行时 Demo、代码目标、来源、Implementation Map 与实施方案一起冻结；不因此减少确认或 Gate。
4. 运行时 Demo：定义整体审美、叙事、光影、构图、动态意图与体验上限。
5. Stitch：提供可编辑的页面结构、尺寸、间距、颜色、字体、组件与内容节点数据。
6. DESIGN.md / Site Contract：提供 token、响应式、无障碍、交互、动效与禁止模式。
7. 设计师可直接调整 Stitch；APEX 同步最新画布并比较冻结差异，重大视觉或交互变化会撤销旧 Gate 2。
8. Stitch 使用同一 Demo 的截图、DOM、来源清单与参数合同建立画布；无原生图片输入时走 Stitch UI 导入，否则报告能力缺口，禁止静默退化为纯文本。仅当用户明确要求“跳过 Stitch 步骤”时，Router 才可受控地跳过该阶段并锁定已登记 Demo；“继续”或普通 skip 都不会触发该路径。

## 7. 严格视觉到代码链路

所有路线先将运行时 Demo 作为共同视觉基线；选择 Stitch 时，运行时 Demo、Stitch 画布与代码是同一参数化视觉合同的物化结果；选择直接代码时，已登记并被路线选择接受的运行时 Demo、来源与实施方案构成可审计的代码基线。两者都不是人工目测，而是以结构、内容、token 和截图证据共同约束。

1. Gate 1 后先确认完整视觉描述与实施方案；方案确认后自动生成并登记运行时 Demo 与参数合同，锁定中文内容、表格、布局节点、图表编码、组件和 design token，不再增加 Demo 确认。
2. Stitch Freeze 保存确认 Screen 的 HTML、完整截图、内容哈希和 generation input；strict-replica 完成运行时 Demo 到 Stitch 的结构与保真校验后才 Seal。

3. 运行时由真实浏览器采集截图与 DOM。运行时 Demo 路线完成 Stitch 到代码的第二段结构和视觉校验；直接代码路线验证其确认描述、来源、映射和运行时结果一致。任何结构、内容或设计 token 差异均不能以人工近似放行。
4. ``data-apex-*` 标记锁定布局、图表、组件和 token；哈希不一致、标记缺失或关键差异均不得进入下一 Gate。

## 7.1 全元素来源锁定与可追溯实施

APEX 3.22 将运行时 Demo 中的视觉元素从“视觉提示”升级为可复用、可验证的来源合同，并在运行时 Demo 前加入 Visual Execution Plan 确认。任何需要落代码的元素都必须在实施前锁定其具体来源，代码阶段不得重新检索相似元素或用近似填充替代。

1. Visual Source Manifest 为每个视觉节点锁定布局、组件、样式和文字内容来源；图标、素材、图表、图示与动效出现时还需锁定资源 ID、版本和参数。
2. 来源可来自项目既有资产、宿主 Skill、APEX 官方注册表、原生 Web 能力、用户提供内容或经确认的新增依赖；宿主 Skill、官方注册来源和新增依赖锁会被机器校验。
3. Motion Capability Inventory 与 Selection 在运行时 Demo 生成前盘点现有动效资产、已安装库和可用专业能力；动效预览、Motion Contract 和实现映射必须使用同一引擎、API、关键帧和时间参数。
4. Implementation Map  
消费同一份来源清单，并记录代码目标、选择器、实现方式和来源标记。实施代码必须保留 `data-apex-source` 标记，审计会验证标记真实出现在对应文件中。
5. Gate 2 验证来源清单、Visual Bundle、动效合同、依赖锁和实现映射；Gate 3  
验证真实代码仍保留同一来源标记与运行时证据。未登记来源、未锁版本或相似替代均不能通过。
6. 明确跳过 Stitch 阶段会写入 `stage-skip` 决策回执，不会伪造 Stitch Freeze 或放宽 Gate。Visual Bundle、实现对比证据和运行状态矩阵都必须绑定已登记并由路线选择接受的运行时 Demo；普通 ``skip stitch`` 仍只可豁免人工确认，必须已有真实 Stitch 工件。
7. 运行时 Demo 前先展示并确认 Visual Execution Plan：布局、颜色、字体、内容、组件、特效、动效引擎/API、在线候选和待安装依赖。未安装候选只有同时进入确认方案和精确 Dependency Lock 后才可进入 Gate 2；确认前 Router 不授予正式运行时 Demo 生成。

# APEX 工作台原型：分析、冻结与交付

以下为 APEX

对用户和交付团队的典型工作台原型。它们不是某个业务系统的页面，而是将需求、视觉和证据在同一 run 内连接起来的操作模型。

| 分析工作台                                                                 | 视觉冻结台                                                                           | 交付验收台                                                                     |
|-----------------------------------------------------------------------|---------------------------------------------------------------------------------|---------------------------------------------------------------------------|
| 用户意图：专业、高级的成长报告<br>已提炼：目标 / 假设 / 风险<br>待确认：业务范围与质量线<br>当前 Gate：Gate 1 | Visual：方案确认后自动生成 Demo<br>Demo 登记后：Stitch 或直接代码<br>Stitch：单独生成与单独确认<br>当前阶段：视觉冻结 | 实现映射：组件 / API / 权限<br>真实证据：浏览器 / 契约 / 性能<br>回归状态：页面家族通过<br>当前 Gate：Gate 3 |
| 下一步：受控推进                                                              | 下一步：受控推进                                                                        | 下一步：受控推进                                                                  |

三个工作台共享同一项目 run、同一视觉事实和同一验收合同；用户在关键 Gate 确认，APEX 才会向下一阶段授权。

## 8. 前后端工程实施与技术选择

Gate 2 后才允许实施。`implementation-map.json` 让每个设计节点可追溯到来源、代码目标、token、数据/API、事件、权限、响应式、测试选择器与验收条件。Existing 项目还须验证基线入口与实施目标真实存在。

| 基础交互  | Radix、shadcn/ui、HeroUI 由项目和 Gate 2 选择其一或等价实现；不是强制全局依赖                        |
|-------|------------------------------------------------------------------------------|
| 设计系统  | google-design-md、Site Contract、DESIGN.md、组件状态与 token 约束                      |
| 图标与动效 | Lucide、Motion / Framer Motion、React Bits、motion-ai-kit；需满足性能与 reduced motion |
| 组件验证  | Storybook 状态展示、交互检查、视觉回归与可访问性审查                                              |
| 后端与接口 | Domain Model、API Contract、真实请求/响应样本与契约验证；不得用演示数据替代生产事实                       |

## 9. 数据、趋势、洞察与智能表达

对于成长报告、经营分析等数据产品，APEX 不允许图表只展示孤立结果。每个指标必须在产品合同中明确数据、趋势、洞察、原因、预测和建议六层表达，并将异常、目标、预测及 AI 解读映射为可验证的图表与状态。

1. 数据层说明指标口径、时间范围、来源、权限与缺失处理。
2. 趋势层展示变化方向、速率、对比区间与显著异常。
3. 洞察、原因、预测与建议分别声明证据边界；模型推断不得伪装成确定事实。
4. 图表编码、目标线、预测区间、异常注释、tooltip、空态和降级均写入视觉合同与实施映射。

## 10. 验收、质量证据与一次性落地

一次性落地不是“一次生成就不再验证”，而是首次方案与实施链条能以真实证据闭合，不让用户在中途重复补充已经可从项目事实或已确认冻结中获得的信息。

1. 功能：单元、集成、运行时和关键交互验证。
2. 视觉：真实截图、冻结基线与逐屏批准，不能以“看起来相似”替代。
3. 无障碍：合同指定 WCAG 目标与零遗留违规。

- 4. 性能：LCP、INP、CLS 与测量环境。
- 5. 接口：真实请求/响应样本与 API Contract。
- 6. 回归：响应式、页面家族、Existing 保护边界及整站关键路径。
- 7. 稳定性矩阵：至少两个视口，并记录默认、加载、空、异常、无权限五类状态的通过证据；不适用状态必须留下原因，关键交互必须有真实运行证据。

## 11. Router-first、Session 隔离与受控执行

APEX 4.37.1 将 Markdown 规范与代码化路由分层：Markdown  
描述角色、标准、约束与策略；代码负责不可跳过的  
run、session、Gate、审批、授权、取消、幂等与并发判断。

唯一 Core 位于 `~/codex/apex/APEX`。项目运行产物只在 `/.apex/`，Core 不保存项目截图、运行时 Demo、Stitch、证据、用户资料或缓存。

| 新 session | 必须创建新 run；不得自动复用旧 run                                                  |
|-----------|------------------------------------------------------------------------|
| 恢复        | 仅同一 session 可以恢复自己绑定的 run；跨 session 需显式交接授权                            |
| 自动同步      | 每次 Router 调用先将最新 Core Bridge 发布到全局 apex Skill，再校验哈希；失败即阻断              |
| 授权        | 绑定项目、run、session、Gate、动作、state hash 与短时过期；状态变化立即使旧授权失效                 |
| 取消与重跑     | cancel 保留证据并释放 lease；restart 只在规则允许时继承 Gate 1 事实                       |
| 并发        | mutation lease 阻止不同 run 同时修改；Gate 2 后可使用 FIFO mutation 队列              |
| 隔离实施      | Git 项目可按 run 创建 detached worktree；状态与证据仍留在原项目 .apex                    |
| 执行        | Action Gateway 只执行已登记、已授权且带 Router 守卫的脚本；operation receipt 防止相同动作重复副作用 |
| 人工复核      | edited / rejected 复核与哈希工件独立记录，拒绝不会被误记为批准                               |

## 12. Token、隐私、恢复与流程回归

APEX 以阶段化上下文和结构化 JSON 控制 token：只加载当前 Gate 所需规则，复用项目 `/.apex` 中已确认的事实，而不是在每轮对话重复整份规范。

- Checkpoint 记录阶段、Gate、锁和工件哈希，支持在同 session 内恢复；轨迹验证检查审批、授权、Gate 与取消的先后关系。敏感资料、凭据、生产数据与未授权外部内容不得进入 APEX Core；外部 Stitch 或验证调用遵循最小必要传输和项目授权。
1. 上下文编译器记录来源、哈希和失效范围；变化只让受影响产物失效。
  2. 恢复不允许自动绕过 Gate 或复用跨 session 状态；预算不足时缩小范围或重新定界，不省略验收。
  3. 轨迹验证将 Router 事件转为可复核证据，用于防止流程回归与授权顺序倒置。
  4. 宿主通用 shell/文件工具无法被本地 Skill 平台级拦截；APEX 正常路径用 Router 和 Action Gateway 约束高风险动作。

## 13. 安装、接入、Connector 与 Adapter

APEX 安装到当前用户的唯一 Core `/apex/APEX`，默认即 `~/codex/apex/APEX`；安装器不会覆盖非空 Core，也不会自动下载外部 Skill。安装完成后必须运行 `npm run preflight`，只有真实依赖与当前用户 Bridge 可用时才进入既有 Router 工作流。Connector 提供项目类型接入方式，Adapter 只映射项目事实而不复制 Core。

| 安装与预检                 | <code>npm run install:apex</code> 安装当前用户 Core 与内置 Bridge； <code>npm run preflight</code> 校验 Node、Git、npm、必需 Skill、manifest 与规范路径； <code>npm test</code> 验证便携安装和 Router 合约 |
|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Connector             | generic-web-app、react-saas、vue-admin、vanilla-js-app 等项目类型接入指引                                                                                                             |
| Adapter               | 映射产品真相源、设计真相源、代码入口、页面家族、验证方式与保护边界                                                                                                                                         |
| Skill                 | 提供设计、图标、动效、浏览器、组件和工程增强；不能裁决 Gate 或绕过 Router                                                                                                                               |
| 项目 <code>.apex</code> | 保存该项目的 run、审批、授权、事件、证据、缓存和锁，默认不纳入项目版本控制                                                                                                                                   |

## 14. 安全、隐私与治理边界

白皮书采用分层架构、责任边界、可追溯决策、运行控制和文档修订等通用专业白皮书要素。架构描述以利益相关者关注点和视角为组织原则；当前范围聚焦可靠交付控制面，不新增监控、指标采集、告警、控制台或独立平台服务。

- 1. 最小权限：Router 授权绑定项目、run、session、动作、状态哈希和有效期；代码修改另需 mutation lease。
- 2. 可追溯性：审批、授权、事件、冻结、截图与验证证据位于项目`.apex`运行态，支持审计和恢复。
- 3. 数据保护：敏感资料、凭据、生产数据与未经授权外部内容不得写入 APEX Core；外部传输采用最小必要范围。
- 4. 交付韧性：Checkpoint、失效范围、证据重采集、取消、队列与 Gate 撤销避免在变化后继续使用过期结论。
- 5. 版本治理：Core 行为、协议、Schema 与能力注册的变更才触发 APEX 版本；文档和 PDF 作为同步产物，不单独升 Core 版本。

## 15. 版本发布与自动文档同步

APEX 正式更新必须同时更新受影响的 Core、Schema、Bridge、规范、产品/技术/部署文档与本独立 PDF。该要求不依赖人工记忆：发布审计自动同步全局 Skill、验证 manifest YAML 与入口路径、检查 Router 合约与动作守卫、禁止文档展示绕过 Router 的命令，并自动重建本 PDF。

- 1. PDF 封面、正文与 PDF 元数据都写入 manifest 版本；版本不一致即发布失败。
- 2. 发布包必须通过隔离的全新用户目录安装契约：Core 与 Bridge 可复制，固定维护者路径为零，外部必需 Skill 缺失时真实阻断并给出来源与安装命令。
- 3. 资源解析器只把 npm 与 URL 视为本地可执行解析；API / MCP 必须由宿主适配器提供，Skill / 官方站点仅作来源元数据，禁止把探测成功伪装成代码已下载。
- 4. 新 session 与既有 session 的下一 APEX Router 调用自动采用最新 Bridge。已在生成中的回复不能由本地文件反向注入，但下次调用不会继续旧规则。
- 5. 发布审计只证明 APEX Core 可用；具体项目仍必须为本次 run 形成真实 Gate 3 证据。

## 16. 参考框架与公开资料

本白皮书采用公开标准和厂商白皮书中的通用组织方式：先说明对象、受众、问题、价值与范围，再给出架构视角、 workflow、控制机制、运营与安全边界、部署与修订信息。以下资料用于校准结构和工程治理，不构成 APEX 或目标项目获得任何认证的声明。

|                         |                             |
|-------------------------|-----------------------------|
| ISO/IEC/IEEE 42010:2022 | 架构描述、利益相关者关注点、视角和模型类别的组织原则。 |
|-------------------------|-----------------------------|

| 来源                                                                   | 与业务连续性管理的关系                                       |
|----------------------------------------------------------------------|---------------------------------------------------|
| ISO/IEC/IEEE 42030:2019                                              | 以利益相关者关注点、目的和价值评估架构的审视方式。                         |
| AWS Well-Architected Framework                                       | 运营卓越、安全、可靠性、性能与持续改进等运行治理维度。                       |
| Microsoft 产品安全白皮书与实施指南                                               | 服务架构、核心流程、身份与访问、数据保护、审计、部署顺序和责任边界的说明方式。           |
| IBM Carbon、W3C WCAG 2.2 / WAI-ARIA APG、Material Adaptive、DTCG、Fluent | 数据表达、无障碍交互、自适应布局、设计 token 与人本体验的策略校准；不复制任何厂商视觉品牌。 |

# 17. 结语与文档修订

APEX 4.37.1 的目标是让专业判断被保留、让用户确认被尊重、让设计和工程共享同一份事实，并以真实证据定义交付完成。无论从 0 到 1，还是 Existing 改造，只有业务合同、视觉冻结、实施映射和验证证据一致，才可称为真正的一次性落地。

本白皮书为当前版本的独立说明。发布审计会自动更新版本相关内容、重建 PDF 并验证其元数据；具体项目的交付完成仍以该项目自己的 Gate 3 证据为准。

| 业务与产品 | 短需求被提炼为可确认范围、假设、风险与质量合同，而非直接变成不可复核的页面。                                                       |
|-------|----------------------------------------------------------------------------------------------|
| 设计与体验 | 所有路线先以确认运行时 Demo建立视觉事实；Stitch 路线再以 Stitch、设计契约与运行时截图扩展，直接代码路线以运行时 Demo、来源、实施方案与运行时截图构成可审计事实。 |
| 研发与交付 | 每项实现可追溯到组件、接口、权限和测试；交付结论必须有浏览器、契约、性能、无障碍与回归证据。                                               |
| 组织与交付 | Router-first 的 run/session/授权/lease/队列/取消机制保障并发隔离、版本同步、恢复和审计，而项目数据始终留在项目边界。                  |